

## MODULE

## UDF

# User-Defined Functions in Power Query M

PL-300 Exam Ready · Advanced M · Senior Proficiency

From first-principles syntax through closures, higher-order functions, recursive design, function libraries, memoisation, dynamic dispatch and type-safe generics. Every pattern needed to pass PL-300 and build professional, reusable M at scale.

© Adapted for Educational Use | Power Query M — Course Reference Manual

## ■ Key Insight

Functions are first-class values in M — they can be stored in variables, passed as arguments, returned from other functions, and collected into records and lists. This single fact underpins every advanced M pattern: ETL function libraries, dynamic dispatch, factory functions, closures, higher-order transforms and memoisation. Mastering UDFs is what separates a Power Query user from a Power Query engineer.

## Before You Begin

- Prerequisites: Modules 1–6. Fluency with `let...in`, `each`, records, lists and `Table.*` functions required.
- Integrates with Module 7 (control flow) and Module 8 (API connectors).
- No external files required — all exercises use inline data or `List.Generate`.

## Module Contents

### Lesson 1

— Function Fundamentals — syntax, parameters, calling conventions, first-class status

### Lesson 2

— Type Annotations — parameter types, return types, nullable, runtime type checking

### Lesson 3

— Optional Parameters — `??`, nullable, record options pattern, defaults-and-overrides

### Lesson 4

— Scope and Closures — variable capture, factory functions, configurable pipelines

### Lesson 5

— Higher-Order Functions — functions as arguments, function composition, pipelines

### Lesson 6

— Recursive Functions — `@`, base cases, depth limits, hierarchy traversal

### Lesson 7

— Function Libraries — ETL query pattern, versioning, cross-query sharing

### Lesson 8

— Advanced Patterns — dynamic dispatch, memoisation, `SafeRun`, `SmartFormat`

### Lesson 9

— PL-300 Patterns Consolidated — all exam-tested UDF scenarios annotated

**Demonstration 1**

— Building a Production ETL Function Library from Scratch

**Demonstration 2**

— Dynamic Dispatch and Type-Safe Generic Transform Pipeline

**Lab UDF**

— Five exercises: Foundations → Types → Closures → Libraries → Advanced

**Module Review**

— Review questions, best practices, common mistakes, quick reference

## LESSON 1

## Function Fundamentals

A function in M is an expression that takes zero or more parameters and returns exactly one value. The body is any M expression — a literal, a let...in, a conditional, a record, a list or a call to another function. There are no statements, no return keyword and no side effects. Functions are values: they can be stored, passed and returned just like numbers or text.

|                         |                                                                          |
|-------------------------|--------------------------------------------------------------------------|
| <b>Anonymous syntax</b> | ( param1, param2 ) => expression                                         |
| <b>Named syntax</b>     | FuncName = ( param ) => expression ← inside a let...in                   |
| <b>Zero parameters</b>  | () => expression – called with ()                                        |
| <b>Body</b>             | Any M expression: literal, let...in, if/then/else, record, list, call    |
| <b>Return value</b>     | The evaluated body – always exactly one value                            |
| <b>First-class</b>      | A function IS a value. Store it, pass it, return it, put it in a record. |

## Function Syntax — Anonymous, Named, Multi-Parameter

```
// Anonymous single-parameter function
( x ) => x * 2

// Named function in a let...in
let
    Double = ( x ) => x * 2,
    Triple = ( x ) => x * 3,
    Result = Double(5) + Triple(4)    // = 10 + 12 = 22
in Result

// Multi-parameter function with let body
ShippingCost = ( weight, zone ) =>
    let
        Base      = if zone = 'EU' then 5.0 else 12.0,
        Surcharge = if weight > 10 then Base * 0.5 else 0.0
    in Base + Surcharge

ShippingCost( 15, 'EU' )           // = 7.5
ShippingCost( 5,  'US' )           // = 12.0
```

## Calling Functions — Four Patterns

### Calling Functions

```
// Pattern 1: Direct call
Double(5)                                // = 10

// Pattern 2: Call via record field
Lib = [ Double=(x)=>x*2, Triple=(x)=>x*3 ],
Lib[Double](5)                           // = 10

// Pattern 3: Pass a function as an argument
Apply = ( fn, value ) => fn( value )
Apply( Double, 7 )                       // = 14
Apply( (x) => x*x, 6 )                   // = 36

// Pattern 4: Call the function returned by another function
MakeAdder = ( n ) => ( x ) => x + n
AddTen = MakeAdder(10)                   // AddTen is a function
AddTen(5)                                // = 15
```

#### ■ How M Does It

A blank Power Query query whose result IS a function makes that function available to every other query in the .pbix by name. This is how shared utility functions are deployed in production models.

each [Col] is shorthand for  $(\_) \Rightarrow \_[\text{Col}]$ . User-defined functions use the same anonymous function syntax — they are the same mechanism.

#### ■ Feynman Check

##### Check your model:

- What does 'first-class values' mean for functions? Give two things you can do with a function that you cannot do with a SQL stored procedure.
- Write IsEven(n) two ways: (a) anonymous expression; (b) named let binding.
- A colleague writes: FuncName = (x) => let R = x\*2 in R. Is this valid? What does FuncName(5) return?

LESSON 2

Type Annotations

Type annotations declare the expected type of each parameter and the return type. They are syntactically optional but professionally mandatory for any function shared across queries or deployed to Dataflows. M checks types at call time — a mismatch raises `Expression.Error` before the body executes. Annotations also appear in the function documentation dialog in the editor.

|                   |                                                                      |
|-------------------|----------------------------------------------------------------------|
| Annotated syntax  | ( param as type ) as returnType => expression                        |
| as type           | Expected type of the parameter. Checked at call time.                |
| as returnType     | Declared return type. Optional but recommended for shared functions. |
| nullable type     | ( param as nullable number ) – accepts the type OR null.             |
| any               | ( param as any ) – accepts any M value. Use for generic functions.   |
| function          | ( fn as function ) – accepts a function value.                       |
| table/list/record | Type keywords for structured values.                                 |

Type Annotations

```
// Fully annotated function
Double = ( n as number ) as number => n * 2
Double(5)           // = 10
Double('hello')     // Expression.Error: type mismatch at call time

// Nullable parameter – accepts the type OR null
SafeDouble = ( n as nullable number ) as nullable number =>
    if n = null then null else n * 2
SafeDouble(null)     // = null (no error)
SafeDouble(5)        // = 10

// Higher-order: 'function' type
ApplyTwice = ( fn as function, x as number ) as number => fn(fn(x))
ApplyTwice( Double, 3 ) // = Double(Double(3)) = 12
```

Runtime Type Checking with Value.Is and Value.Type

**Runtime Type Checking**

```

Value.Is( 42,          type number )      // true
Value.Is( 'hello',     type number )      // false
Value.Is( null,        type number )      // false – null is not a number
Value.Is( null,        type nullable number) // true
Value.Is( {1,2,3},     type list )        // true
Value.Is( #date(2024,1,1), type date )    // true

Value.Type( 42 )       // type number
Value.Type( (x)=>x )   // type function

// Runtime dispatch by type
Describe = ( v as any ) as text =>
    if Value.Is(v, type number) then 'number: ' & Text.From(v)
    else if Value.Is(v, type text) then 'text: ' & v
    else if Value.Is(v, type logical) then if v then 'true' else 'false'
    else if Value.Is(v, type date) then Date.ToText(v, 'yyyy-MM-dd')
    else if v = null then '(null)'
    else Text.From(v)

```

**■ PL-300 Exam**

- Type annotations checked at call time — before the body executes.
- nullable number accepts numbers AND null. number rejects null at call time.
- Value.Is(v, type date) is false for datetime values — handle both separately.
- Value.Is(null, type nullable number) = true.

**■ Feynman Check****Check your model:**

- Write Clamp(n, lo, hi) with full type annotations on all three params and the return type.
- Value.Is(null, type nullable number) — true or false? Explain why.
- (n as number) as number is declared. Caller passes null. What happens, and where?

## LESSON 3

## Optional Parameters and Default Values

M does not support default parameter values in the function signature. Optional parameters are declared with the optional keyword — when the caller omits them M passes null. Inside the function, use the ?? null-coalescing operator to supply the default. For functions with many optional parameters the **record options pattern** is the professional standard: one optional opts record merged with a Defaults record.

|                               |                                                                            |
|-------------------------------|----------------------------------------------------------------------------|
| <b>optional keyword</b>       | Marks a parameter optional. Caller may omit it; M passes null if omitted.  |
| <b>x ?? default</b>           | Null-coalescing: returns x when x is not null, otherwise default.          |
| <b>Record options pattern</b> | optional opts as nullable record – merge with Defaults for many optionals. |
| <b>Record.Combine({A, B})</b> | New record with fields from A and B. B's fields override A's.              |
| <b>opts ?? []</b>             | Handles both omitted opts (null) and empty record [].                      |

## Single Optional Parameter

```
// Single optional parameter with ??
FormatNum = ( n as number, optional decimals as nullable number ) as text =>
    let D = decimals ?? 2
    in Number.ToText( Number.Round(n, D) )

FormatNum( 3.14159 )           // = '3.14'   (default 2)
FormatNum( 3.14159, 4 )       // = '3.1416'
FormatNum( 3.14159, null )    // = '3.14'   (null treated as omitted)
```

## Record Options Pattern

```
// Record options pattern – scalable for many optional parameters
TransformSales = ( src as table, optional opts as nullable record ) as table =>
    let
        Defaults = [ RoundTo=2, RemoveNulls=true, MaxRows=null, MinRev=0 ],
        0         = Record.Combine( { Defaults, opts ?? [] } ),
        Cleaned   = if 0[RemoveNulls]
                    then Table.SelectRows(src, each [Revenue] <> null)
                    else src,
        Filtered  = Table.SelectRows(Cleaned, each [Revenue] >= 0[MinRev]),
        Rounded   = Table.TransformColumns(Filtered,
            { {'Revenue', each Number.Round(_, 0[RoundTo]), type number} } ),
        Limited   = if 0[MaxRows]=null then Rounded
                    else Table.FirstN(Rounded, 0[MaxRows])
    in Limited

TransformSales( RawData )           // all defaults
TransformSales( RawData, [ MaxRows=500 ] ) // one override
TransformSales( RawData, [ MinRev=100, RoundTo=0 ] ) // two overrides
```

## ■ Pro Tip

The record options pattern is extensible without changing the function signature: add one field to Defaults. All existing callers continue working unchanged. This is impossible with individual optional parameters once a function is in use.

### ■ Watch Out

M does not distinguish between 'omitted' and 'explicitly passed as null'. Both appear as null inside the function. '0 ?? default' returns 0 — zero is not null. 'false ?? default' returns false — false is not null. ?? is null-coalescing only.

### ■ Feynman Check

#### Check your model:

- '0 ?? 99' — what does this return? "" ?? "default"?
- You add a Locale parameter to FormatNum without breaking 20 existing callers. Which approach (individual optional vs record options) is safer? Why?
- Record.Combine({Defaults, opts ?? []}) — what happens when opts has a field Defaults does not have?

LESSON 4

Scope and Closures

A **closure** is a function that captures variables from its enclosing lexical scope. In M, any function defined inside a `let...in` automatically captures all visible bindings. The captured value is fixed when the function is created. Because M has no mutable variables there are no shared-state closure bugs. Closures enable factory functions and pre-configured pipelines.

Basic Closure

```
// Basic closure – IsHighValue captures 'Threshold' from the outer let
let
    Threshold      = 5000,
    IsHighValue    = ( amount as number ) => amount >= Threshold, // closure
    Revenues       = { 1200, 6500, 3400, 8900, 500 },
    HighRevenues   = List.Select( Revenues, IsHighValue )
in HighRevenues           // = { 6500, 8900 }
```

Factory Function

```
// Factory function – returns a NEW closure capturing its parameter
MakeMultiplier = ( factor as number ) as function =>
    ( x as number ) as number => x * factor // 'factor' is captured

Double = MakeMultiplier(2)           // factor = 2 baked in
Triple = MakeMultiplier(3)           // factor = 3 baked in
Double(7)                             // = 14
Triple(7)                             // = 21
MakeMultiplier(10)(5)                // = 50 – call immediately
```

Practical Factory — Date Filter Generator

```
// Practical factory: date-range filter generator
MakeDateFilter = ( startDate as date, endDate as date ) as function =>
    ( src as table, dateCol as text ) as table =>
        Table.SelectRows( src,
            each Record.Field( _, dateCol ) >= startDate
            and Record.Field( _, dateCol ) <  endDate )

Filter2024 = MakeDateFilter( #date(2024,1,1), #date(2025,1,1) )
Filter2024( FactSales, 'SaleDate' ) // applies pre-configured filter
Filter2024( FactReturns, 'ReturnDate' ) // reuse on a different table
```

|                               |                                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------|
| Rule 1: Lexical scoping       | A function sees all names in the enclosing <code>let...in</code> at definition time. |
| Rule 2: Capture at definition | The captured value is fixed when the function is created, not called.                |
| Rule 3: No mutation           | M has no mutable variables – no shared-state closure bugs.                           |
| Rule 4: Nested capture        | Inner functions can capture from all outer scopes, chaining through levels.          |

#### ■ Feynman Check

##### Check your model:

- `MakeMultiplier(3)(7)` — trace evaluation. What is captured? When? What is returned at each step?
- Two calls to `MakeMultiplier(2)` — are the returned functions the same object? Do they share state?
- Write `MakeRangeFilter(lo, hi)` returning a function that selects list elements between `lo` and `hi`.

## LESSON 5

## Higher-Order Functions

A higher-order function takes another function as an argument, returns a function as its result, or both. M's built-in table and list functions are all higher-order — `Table.AddColumn`, `List.Transform`, `List.Select` all take a function argument. Writing your own lets you build generic, composable abstractions that work with any logic the caller provides.

## ApplyToAll and Compose

```
// Takes a function as argument
ApplyToAll = ( lst as list, fn as function ) as list =>
    List.Transform( lst, fn )
ApplyToAll( {1,2,3,4,5}, each _ * 2 ) // = {2,4,6,8,10}

// Returns a function (Compose: apply f first, then g)
Compose = ( f as function, g as function ) as function =>
    ( x as any ) => g( f(x) )

Double = (x) => x*2, AddOne = (x) => x+1,
DoubleThenAdd = Compose( Double, AddOne ) // AddOne( Double(x) )
DoubleThenAdd(5) // = AddOne(10) = 11
```

## Pipeline — Compose a List of Functions

```
// Pipeline: compose a list of functions left-to-right
Pipeline = ( fns as list ) as function =>
    List.Accumulate( fns,
        (x) => x, // identity as starting value
        (composed, fn) => Compose(composed, fn)
    )

Transform = Pipeline({
    each _ * 2, // step 1: double
    each _ + 10, // step 2: add 10
    each _ * _ // step 3: square
})
Transform(3) // = ((3*2)+10)^2 = 256
```

## Higher-Order — Generic Column Validator

```
// Generic column validator – predicate passed as argument
ValidateCol = ( src as table, col as text, pred as function ) as table =>
    Table.SelectRows( src, each pred( Record.Field(_, col) ) )

ValidateCol( Sales, 'Revenue', each _ > 0 ) // positive only
ValidateCol( Sales, 'Region', each _ <> null ) // non-null only
ValidateCol( Sales, 'Date', each _ >= #date(2020,1,1) )
```

## ■ The Pattern

The Compose + Pipeline pattern separates the definition of what transformations to apply from the act of applying them. Store steps in a list, compose them into one function with `Pipeline()`, apply to any value. Each step is independently testable.

### ■ Feynman Check

#### Check your model:

- $\text{Compose}(f, g)(x)$  — which is applied first? Write a concrete example.
- Write  $\text{ApplyIf}(\text{lst}, \text{predicate}, \text{fn})$  — applies  $\text{fn}$  only to elements where  $\text{predicate}$  is true, leaving others unchanged.
- Trace  $\text{Pipeline}(\{f, g, h\})$  state after each  $\text{List.Accumulate}$  iteration with  $f(x) \Rightarrow x*2$ ,  $g(x) \Rightarrow x+1$ ,  $h(x) \Rightarrow x*x$ .

## LESSON 6

## Recursive Functions

A recursive function calls itself to solve a smaller version of the same problem. The `@` operator provides the self-reference inside a `let` binding. Without `@` the name being defined is not in scope within its own body. Every recursive function requires a base case (returns without recursing) and a recursive case (calls `@FuncName` with simpler input). Always include a depth limit — circular data causes infinite recursion without one.

|                       |                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------|
| <b>Syntax</b>         | <code>let F = (params) =&gt; if baseCond then baseVal else @F(simplerArgs) in F</code>                   |
| <b>@FuncName</b>      | Self-reference inside a <code>let</code> binding. Required. Cannot use the name without <code>@</code> . |
| <b>Base case</b>      | Returns a value without recursing. Must always be reachable.                                             |
| <b>Recursive case</b> | Calls <code>@F</code> with input strictly closer to the base case.                                       |
| <b>Depth limit</b>    | <code>if depth &gt; N then error '...'</code> . Protects against circular data.                          |
| <b>Alternative</b>    | For <code>depth &gt; 100</code> or iteration, use <code>List.Generate</code> — no stack overflow risk.   |

## Factorial and GCD

```
// Factorial – base: n<=1 = 1. Recursive: n * (n-1)!
let Factorial = (n as number) as number =>
    if n < 0 then error Error.Record('Arg', 'n must be >= 0', n)
    else if n <= 1 then 1
    else n * @Factorial(n - 1)           // @ required
in Factorial(6)                        // = 720

// GCD – base: b=0. Recursive: GCD(b, a mod b).
let GCD = (a as number, b as number) as number =>
    if b = 0 then a else @GCD(b, Number.Mod(a, b))
in GCD(48, 18)                        // = 6
```

## Recursive List Flatten

```
// Flatten nested list – any depth
let Flatten = (lst as list) as list =>
    List.Combine(List.Transform(lst,
        (item) => if Value.Is(item, type list)
                    then @Flatten(item) else {item}))
in Flatten({{1,2},{3,{4,5}},6})       // = {1,2,3,4,5,6}
```

Hierarchy Path with Depth Limit

```
// Hierarchy path – ALWAYS include depth limit
let
    Buf      = Table.Buffer(HierarchyTable), // buffer prevents N re-queries
    GetPath = (nodeID as text, depth as number) as text =>
        let
            Row  = Table.SelectRows(Buf, each [NodeID] = nodeID),
            PID  = if Table.IsEmpty(Row) then null else Row[ParentID]{0}
        in
            if PID = null or depth > 20 // base: root OR depth limit
            then nodeID
            else @GetPath(PID, depth+1) & ' > ' & nodeID,
    Result = Table.AddColumn(Buf, 'Path',
        each @GetPath([NodeID], 0), type text)
in Result
```

| Problem                         | Best Approach                                              |
|---------------------------------|------------------------------------------------------------|
| Tree / hierarchy traversal      | Recursion with @. Include depth limit.                     |
| Math reduction (GCD, factorial) | Recursion with @. Depth typically < 50.                    |
| Nested list/record flattening   | Recursion with @. Depth = nesting depth.                   |
| Pagination / counters           | List.Generate. Iterative. No stack overflow.               |
| Retry loops                     | Recursive with attempt counter. Depth limit = max retries. |
| Deep trees (> 100 levels)       | List.Generate with explicit stack. Avoids overflow.        |

■ Feynman Check

Check your model:

- Why does M require @? What error would you get without it?
- Circular hierarchy A→B→A. Depth limit = 20. What does GetPath return for A?
- Naive recursive Fibonacci makes 2^n calls. What is the M-efficient alternative?



#### ■ Pro Tip

Add a Help record to the library for self-documentation: `Help = [SafeDate = 'SafeDate(v) → nullable date. Returns null on bad input.', ...]`. Accessible as `ETL[Help][SafeDate]` from any query.

Disable the ETL query from model load: right-click → Properties → uncheck Load enabled. It is a staging query — it must NOT appear as a model table.

#### ■ Feynman Check

##### **Check your model:**

- Name three maintenance advantages of a function library over copy-pasted functions.
- Can `ETL[ValidateSchema]` call `ETL[SafeDate]` inside its body? What constraint applies?
- 'Load to model' is accidentally enabled on ETL. What appears in the Power BI Fields pane?

## LESSON 8

## Advanced UDF Patterns

**Pattern 1: Dynamic Dispatch with Function Records**

Store functions keyed by name in a record. Select and call at runtime using `Record.FieldOrDefault`. No if/else chains.  $O(1)$  lookup. Extensible by adding one record field.

**Dynamic Dispatch**

```
Formatters = [
    Currency = (n) => '$' & Text.From(Number.Round(n,2)),
    Percent  = (n) => Text.From(Number.Round(n*100,1)) & '%',
    Integer  = (n) => Text.From(Number.RoundDown(n,0)),
    Default  = (v) => Text.From(v)
],
FormatType = 'Currency', // from config or parameter
ApplyFmt    = Record.FieldOrDefault(Formatters, FormatType, Formatters[Default]),
Result = Table.AddColumn(Sales, 'Fmt', each ApplyFmt([Revenue]), type text)
```

**Pattern 2: Memoisation — Cache Computed Results**

Pre-compute all needed values into a buffered lookup table. Subsequent lookups read from RAM instead of recomputing.

**Memoised Fibonacci**

```
// Fibonacci: pre-compute n=0..50 with List.Generate, buffer once
FibSeq = List.Buffer(
    List.Skip(List.Generate(
        () => [a=0, b=1],
        each [a] <= 12586269025,
        each [a]=[b], b=[a]+[b]],
        each [a] ), 0) ),

FibLookup = (n as number) as number =>
    if n < 0 or n >= List.Count(FibSeq)
    then error Error.Record('Arg', 'n out of range', n)
    else FibSeq{n}

List.Transform({0..10}, FibLookup) // = {0,1,1,2,3,5,8,13,21,34,55}
```

**Pattern 3: Generic SafeRun Error-Handling Wrapper**

**SafeRun — Generic Error Wrapper**

```
// Run any function safely – return structured result record
SafeRun = (fn as function, arg as any) as record =>
    let attempt = try fn(arg)
    in if attempt[HasError]
        then [ OK=false, Value=null, Err=attempt[Error][Message] ]
        else [ OK=true, Value=attempt[Value], Err=null ]

SafeRun(Date.From, '2024-01-15') // [OK=true, Value=#date(2024,1,15), Err=null]
SafeRun(Date.From, 'not-a-date') // [OK=false, Value=null, Err='...']

// Apply across a column – isolate per-row errors
WithResult = Table.AddColumn(Source, 'DateResult',
    each SafeRun(Date.From, [RawDate]), type record)
```

**Pattern 4: SmartFormat — Type-Safe Generic Transformer****SmartFormat — Type-Safe Generic**

```
// Handles any M type – use on any column without knowing its type
let SmartFormat = (v as any) as text =>
    if v = null then '(null)'
    else if Value.Is(v, type number) then Number.ToText(v)
    else if Value.Is(v, type date) then Date.ToText(v, 'yyyy-MM-dd')
    else if Value.Is(v, type datetime) then DateTime.ToText(v, 'yyyy-MM-dd HH:mm')
    else if Value.Is(v, type logical) then (if v then 'Yes' else 'No')
    else if Value.Is(v, type list) then
        '[' & Text.Combine(List.Transform(v, each @SmartFormat(_)), ',') & ']'
    else Text.From(v)
in SmartFormat

// Apply to every column – fully dynamic
Table.TransformColumns(MixedTable,
    List.Transform(Table.ColumnNames(MixedTable),
        (col) => {col, SmartFormat, type text}))
```

**■ PL-300 Exam**

- Dynamic dispatch: Record.FieldOrDefault(Dispatchers, key, defaultFn) — no if/else, O(1), extensible.
- Memoisation: pre-compute into Table.Buffer or List.Buffer. Look up by key. Single evaluation.
- SafeRun: try fn(arg) returns [HasError, Value/Error]. Catches all errors from any function.
- SmartFormat: Value.Is(v, type date) is false for datetime — handle both.

**■ Feynman Check****Check your model:**

- Adding a new formatter to the Formatters dispatch table — how many lines of code change?
- Why is List.Buffer used in the memoised Fibonacci? What happens without it?
- SafeRun(fn, arg) — if fn itself is null, what happens? Add a guard.

## LESSON 9

## PL-300 Patterns Consolidated

The PL-300 exam tests UDFs in three areas: correct function syntax and calling conventions, reusable transformation patterns, and function composition with table operations. All five patterns below are annotated exam solutions.

### Pattern 1: Reusable Custom Column Logic

#### Pattern 1 — Reusable Tier Logic

```
let
    AssignTier = (rev as nullable number) as text =>
        if rev = null then 'Unknown'
        else if rev >= 100000 then 'Platinum'
        else if rev >= 50000 then 'Gold'
        else if rev >= 10000 then 'Silver'
        else 'Bronze',
    WithTier = Table.AddColumn(FactSales, 'Tier',
        each AssignTier([Revenue]), type text),
    ReturnsTier = Table.AddColumn(FactReturns, 'Tier',
        each AssignTier([Refund]), type text)
in WithTier
```

### Pattern 2: Parameterised Date Filter

#### Pattern 2 — Parameterised Date Filter

```
DateFilter = (src as table, dateCol as text,
    startDate as date, endDate as date) as table =>
    Table.SelectRows(src,
        each Record.Field(_, dateCol) >= startDate
        and Record.Field(_, dateCol) < endDate)

// Works on any table with any date column
DateFilter(FactSales, 'SaleDate', #date(2024,1,1), #date(2025,1,1))
DateFilter(FactReturns, 'ReturnDate', #date(2024,6,1), #date(2024,7,1))
```

### Pattern 3: Comprehensive Validation Function

**Pattern 3 — Validation Function**

```

ValidateSource = (src as table) as table =>
    let
        Required = {'CustomerID', 'Date', 'Revenue'},
        Missing = List.Difference(Required, Table.ColumnNames(src)),
        NullKeys = Table.RowCount(Table.SelectRows(src, each [CustomerID]=null)),
        Errors = List.Select({
            if List.IsEmpty(Missing) then null
            else 'Missing: ' & Text.Combine(Missing, ', '),
            if NullKeys = 0 then null
            else 'Null keys: ' & Text.From(NullKeys)},
            each _ <> null)
    in
    if List.IsEmpty(Errors) then src
    else error Error.Record('Validation.Failed',
        Text.Combine(Errors, ' | '), [FailCount=List.Count(Errors)])

```

**Pattern 4: Recursive Hierarchy Flattener****Pattern 4 — Hierarchy with Path and Level**

```

let
    Buf = Table.Buffer(HierarchyTable),
    GetAncs = (nodeID as text, depth as number) as list =>
        let
            Row = Table.SelectRows(Buf, each [NodeID] = nodeID),
            PID = if Table.IsEmpty(Row) then null else Row[ParentID]{0}
        in
            if PID = null or depth > 20
            then {nodeID}
            else @GetAncs(PID, depth+1) & {nodeID},
    WithPath = Table.AddColumn(Buf, 'Path',
        each Text.Combine(@GetAncs([NodeID],0), ' > '), type text),
    WithLevel = Table.AddColumn(WithPath, 'Level',
        each List.Count(Text.Split([Path], ' > '))-1, type number)
in WithLevel

```

**Pattern 5: Function as Runtime Aggregation Parameter****Pattern 5 — Function as Runtime Parameter**

```

AggregateCol = (src as table, keyCol as text,
    valCol as text, aggFn as function) as table =>
    Table.Group(src, {keyCol},
        {'AggValue', each aggFn(Record.Field(_, valCol)), type number}))

AggregateCol(Sales, 'Region', 'Revenue', List.Sum )
AggregateCol(Sales, 'Region', 'Revenue', List.Average)
AggregateCol(Sales, 'Region', 'Revenue', List.Max )

// Dynamic: choose aggregation from config
AggMap = [Sum=List.Sum, Avg=List.Average, Max=List.Max, Min=List.Min],
SelFn = Record.FieldOrDefault(AggMap, Config[Method], List.Sum),
Result = AggregateCol(Sales, 'Region', 'Revenue', SelFn)

```

#### ■ PL-300 Exam

- Every PL-300 UDF question tests: correct syntax (parentheses, =>), type annotations, ?? operator, and composing with Table.AddColumn / List.Transform / Table.Group.
- Record.Field(\_, colName) enables dynamic column access when the column name is a variable.
- Functions in a library record are the correct answer to 'sharing logic across multiple queries'.
- For hierarchy patterns: always state the depth limit and explain what it prevents.

**DEMONSTRATION 1****Building a Production ETL Function Library from Scratch**

**Scenario:** Seven queries share duplicate transformation logic. Two developers updated the same copy-pasted function independently and the versions diverged. Goal: consolidate into a versioned library and prove a single edit propagates everywhere.

**Step 1.** Audit the seven queries. List every duplicated function. Count total duplicated lines.

**Step 2.** Create blank query 'ETL'. Define Version='1.0', Help record, and all shared functions with full type annotations.

**Step 3.** Test the library: call ETL[SafeDate]('bad') → null. ETL[NormaliseText](' hello world ') → 'Hello World'.

**Step 4.** Replace the duplicated step in QueryA with ETL[SafeDate](...). Refresh. Verify output is identical.

**Step 5.** Replace all seven queries. Count before/after line counts. Applied Steps pane is shorter, output identical.

**Step 6.** Change Gold threshold from 50 000 to 75 000 in ETL[AssignTier]. Refresh all seven — all pick up the change automatically.

**Step 7.** Trigger ETL[ValidateSchema] with a missing column. Show Reason, Message and Detail in Power BI's error dialog.

**Step 8.** Update Version to '1.1'. Show ETL[Version] = '1.1' from every downstream query — without touching them.

**■ RESULT**

Seven queries reference ETL — no duplicated logic anywhere.

Tier threshold change: one ETL edit → all queries update on next refresh.

Line count reduced 60–75% across seven queries.

ETL[Version] = '1.1' — visible from any downstream query.

**DEMONSTRATION 2****Dynamic Dispatch and Type-Safe Generic Transform Pipeline**

**Scenario:** Five source systems use different date formats, numeric precision and text casing. Goal: one configurable pipeline — no if/else chains, no duplicate queries.

**Step 1.** Define SourceConfig: one row per source with DateFormat, NumDecimals, TextCase columns.

**Step 2.** Build Formatters record: Proper=Text.Proper, Upper=Text.Upper, Lower=Text.Lower, None=(t)=>t.

**Step 3.** Build DateParser record: ISO=(v)=>Date.From(v), US=(v)=>Date.FromText(v,'en-US'), EU=(v)=>Date.FromText(v,'en-GB').

**Step 4.** Build ConfiguredPipeline(sourceName) → looks up config row → selects correct functions via Record.FieldOrDefault → returns (src as table) as table.

**Step 5.** Apply: ConfiguredPipeline('SystemA')(SourceA\_Data), etc. Verify each source is transformed correctly.

**Step 6.** Apply SmartFormat to combined output — all columns to text, all types handled dynamically.

**Step 7.** Add new source row to SourceConfig. ConfiguredPipeline(newSource) works immediately — zero M code changes.

**■ RESULT**

Five sources processed with their respective configurations.

New source: add one SourceConfig row, no M code changes.

New formatter: add one Formatters field, no pipeline code changes.

Zero if/else chains in any dispatch logic.

## LAB UDF

## User-Defined Functions in Power Query M

## ■ How M Does It

Advanced Editor only — no ribbon clicks. Write all M by hand. Complete in order.

- Step 1 — Explain It: answer questions before touching the editor.
- Step 2 — Build It: construction tasks with hints and expected results.
- Step 3 — Break It: deliberate failures — read error, fix it.
- Step 4 — Teach It Back: reflection without consulting the lesson.

No solution code is provided. Hints are structured clues, not answers.

## Exercise 1 — Function Fundamentals and Type Annotations

■ 25 minutes

STEP 1  
Explain It First

## ■ Feynman Check

Answer before opening the editor:

- What does 'first-class value' mean for a function? Give two concrete examples.
- A function is declared (n as number) as number. Caller passes the text '5'. What happens and when?
- Value.Is(null, type nullable number) — true or false? Explain.
- Write IsEven(n) two ways: (a) anonymous expression; (b) named let binding.

STEP 2  
Build It

## Task 1.1 — Five Utility Functions

Create blank query Ex1\_Functions. Define and test all five in one let...in.

- **Clamp(n, lo, hi)**: Returns n clamped to [lo, hi]. Annotate all params and return as number.
- **IsWeekend(d)**: True if Saturday or Sunday. Param: date. Return: logical.
- **RoundSF(n, sigFigs)**: Rounds n to sigFigs significant figures. Hint: use Number.Log10 for magnitude.
- **TruncateText(t, maxLen)**: Returns t trimmed to maxLen chars, appending '...' if truncated.
- **SafeDivide(a, b, opts)**: Divides a by b. opts: nullable record with OnZero field (default null). Returns OnZero when b=0.

■ **Hint** *Clamp: if n < lo then lo else if n > hi then hi else n*

■ **Hint** *IsWeekend: Date.DayOfWeek(d, Day.Monday) — Sat=5, Sun=6. Weekdays are 0–4.*

■ **Hint** *RoundSF: magnitude = Number.RoundDown(Number.Log10(Number.Abs(n)),0). Round to -(magnitude)+sigFigs-1.*

■ **Hint** *TruncateText: if Text.Length(t) > maxLen then Text.Start(t, maxLen) & '...' else t*

■ **Hint** *SafeDivide: O = Record.FieldOrDefault(opts ?? [], 'OnZero', null). if b = 0 then O else a/b*

**■ RESULT**

Clamp(15,0,10)=10. Clamp(-5,0,10)=0. Clamp(7,0,10)=7.  
 IsWeekend(#date(2025,1,4))=false(Sat). IsWeekend(#date(2025,1,5))=true(Sun). Verify the actual day first.  
 RoundSF(123456,3)=123000. RoundSF(0.001234,2)=0.0012.  
 TruncateText('Hello World',7)='Hello W...'. TruncateText('Hi',10)='Hi'.  
 SafeDivide(10,0,[OnZero=0])=0. SafeDivide(10,0,null)=null. SafeDivide(10,2,null)=5.

**Task 1.2 — Describe Applied to a Mixed-Type Table**

Write your own Describe(v) function (do not copy from the lesson). Apply it to every column of an inline #table using Table.TransformColumns. The table must include: a number, text, date, logical and null column.

**■ Hint**

```
Table.TransformColumns(t, List.Transform(Table.ColumnNames(t), (c) => {c, Describe, type text}))
```

**■ RESULT**

All columns converted to text with type prefix.  
 Number → 'number: 42'. Date → 'date: 2024-01-15'. Logical → 'true'. Null → '(null)'.

**STEP 3  
Break It**

- Call Clamp with text for n. Read the exact error. Does it occur before or inside the body?
- Declare SafeDivide with b as number (not nullable). Pass b=0. Error at call time or inside? Why?
- Write a function with no type annotations. Call with a number, then text, then record. What does this reveal?

**STEP 4  
Teach It Back****DEFINITION**

Explain: 'What is the difference between type annotations on a function parameter and checking type inside the function with Value.Is?' Include: when each is appropriate and what happens when a caller passes the wrong type under each approach.

**Exercise 2 — Optional Parameters, Closures, Factory Functions**

■ 25 minutes

**STEP 1  
Explain It First****■ Feynman Check**

Before writing code:

- 'false ?? 99'? '0 ?? 99'? '"" ?? "default"'? Explain the rule.
- Complete: 'A closure is a function that \_\_\_\_\_ from its \_\_\_\_\_ scope.'
- MakeAdder(10)(5) — trace the evaluation. What is captured? When?
- Why is the record options pattern better than individual optional params for extensibility?

**STEP 2  
Build It****Task 2.1 — FormatCurrency with Record Options**

Write `FormatCurrency(amount, optional opts)`. `opts` supports: `Decimals` (default 2), `Symbol` (default '\$'), `SymbolPosition` ('before'/'after', default 'before'). Test all defaults and each override individually.

■ **Hint** `Defaults = [Decimals=2, Symbol='$', SymbolPosition='before']`

■ **Hint** `O = Record.Combine({Defaults, opts ?? []})`

■ **Hint** `Formatted = Number.ToText(Number.Round(amount, O[Decimals]))`

■ **Hint** `Position: if O[SymbolPosition]='before' then O[Symbol]&Formatted else Formatted&O[Symbol]`

#### ■ RESULT

`FormatCurrency(1234.5) = '$1234.50'`

`FormatCurrency(1234.5,[Symbol='€',SymbolPosition='after']) = '1234.50€'`

`FormatCurrency(1234.5,[Decimals=0]) = '$1235'`

`FormatCurrency(null) = null` (null guard required).

## Task 2.2 — MakeDateFilter Factory Function

Write `MakeDateFilter(startDate, endDate)` returning a function (`src` as table, `dateCol` as text) as table that filters `src` to rows where the named column falls in `[startDate, endDate)`. Test on two tables with different date columns.

■ **Hint** `Return: (src, dateCol) => Table.SelectRows(src, each Record.Field(_,dateCol)>=startDate and Record.Field(_,dateCol)...`

■ **Hint** `Record.Field(_, colName)` — dynamic column access when the column name is a variable.

#### ■ RESULT

`Filter2024 = MakeDateFilter(#date(2024,1,1), #date(2025,1,1))`

`Filter2024(FactSales,'SaleDate')` — only 2024 rows.

`Filter2024(FactReturns,'ReturnDate')` — same filter, different table.

## Task 2.3 (Stretch) — Compose and Pipeline

Write `Compose(f,g)` where `f` is applied first, then `g`. Write `Pipeline(fns)` that composes a list left-to-right. Verify: `Pipeline({each _*2, each _+10, each _*_})(3) = 256`.

■ **Hint** `Compose(f,g) = (x) => g(f(x))` — `f` is applied first.

■ **Hint** `Pipeline = List.Accumulate(fns, (x)=>x, (composed, fn) => Compose(composed, fn))`

#### ■ RESULT

`Compose(Double, AddOne)(5) = AddOne(Double(5)) = AddOne(10) = 11.`

`Pipeline({each _*2, each _+10, each _*_})(3) = ((3*2)+10)^2 = 256.`

### STEP 3 Break It

- Task 2.1: pass `opts=[Decimals='two']` (text). Where does the error occur? How do you guard against it?
- Task 2.2: call `Filter2024(FactSales,'NonExistentColumn')`. What happens? How does `Record.Field` handle missing fields?

- Write a function capturing a let binding. Change the binding after the function is defined. Does the function see the new value? What does this prove?

#### STEP 4 Teach It Back

##### DEFINITION

A colleague needs to add Locale to FormatCurrency without breaking 20 existing callers. Explain in plain English why the record options pattern solves this, and what would happen with individual optional parameters instead.

### Exercise 3 — Recursive Functions and Higher-Order Patterns

■ 30 minutes

#### STEP 1 Explain It First

##### ■ Feynman Check

Before writing code:

- State the two mandatory parts of every recursive function without using 'base case' or 'recursive case'.
- Why does M require @? What error would you get without it?
- Trace Flatten({1,{2,{3,4}},5}) step-by-step on paper.
- ApplyIf(lst, predicate, fn) — describe in plain English before writing any code.

#### STEP 2 Build It

### Task 3.1 — Three Recursive List Functions

In Ex3\_Recursion, implement all three without using the equivalent built-in function.

- **SumList(lst)**: Sum all numbers recursively. Base: empty=0. — test: SumList({1,2,3,4,5})=15
- **MaxList(lst)**: Maximum value. Base: single element. — test: MaxList({3,1,4,1,5,9,2})=9
- **CountIf(lst,fn)**: Count elements where fn is true. Base: empty=0. — test: CountIf({1..10}, each \_>5)=5

■ **Hint** *SumList: if List.IsEmpty(lst) then 0 else List.First(lst) + @SumList(List.Skip(lst,1))*

■ **Hint** *MaxList: if List.Count(lst)=1 then List.First(lst) else let r=@MaxList(List.Skip(lst,1)) in if List.First(lst)>r then List.First(lst) ...*

■ **Hint** *CountIf: if List.IsEmpty(lst) then 0 else (if fn(List.First(lst)) then 1 else 0) + @CountIf(List.Skip(lst,1),fn)*

##### ■ RESULT

SumList({1,2,3,4,5}) = 15.

MaxList({3,1,4,1,5,9,2}) = 9.

CountIf({1..10}, each \_ > 5) = 5.

All use @ for self-reference. All have explicit stopping conditions.

### Task 3.2 — Hierarchy with Level and Path

Create an inline hierarchy table (NodeID text, ParentID nullable text, Name text) with 8+ nodes across 4 levels. Buffer it. Implement GetPath(nodeID, depth) recursively. Add AncestorPath (text) and Level (number, 0=root)

columns.

■ **Hint** `Buf = Table.Buffer(HierarchyTable)`. Always buffer before recursive row-by-row reference.

■ **Hint** `GetPath` base cases: `ParentID=null` (root) OR `depth>20` (safety limit).

■ **Hint** `Level = List.Count(Text.Split([AncestorPath], '>')) - 1`

#### ■ RESULT

Root node: `AncestorPath` = its own `Name`. `Level` = 0.

Leaf at depth 3: 'Root > Child > Grandchild > Leaf'. `Level` = 3.

Circular reference at depth > 20: partial path, no crash.

### Task 3.3 — Higher-Order Column Validator

Write `ValidateCols(src, rules)` where `rules` is a list of records [`Column='name'`, `Pred=(v)=>logical`, `Msg='error text'`]. Returns a table of rows failing any rule, with a `FailedRules` list column.

■ **Hint** Per row: `List.Select(rules, each not [Pred](Record.Field(row,[Column])))` gives failed rules.

■ **Hint** Filter to rows where `List.IsEmpty(failed rules)` is false after adding the `FailedRules` column.

#### ■ RESULT

Returns only rows with at least one rule failure.

`FailedRules` contains a list of `Msg` strings for each failing row.

Clean rows excluded. Adding a rule: add one record to `rules` — no other changes.

### STEP 3 Break It

- `SumList({})` — what does the base case return? `SumList({null,1,2})` — what happens if the list has null?
- Remove the depth limit from `GetPath`. Add `A→B→A` circular reference. What exact error does M raise? After how many levels?
- `ValidateCols` with a rule whose `Column` does not exist in `src`. What happens? Add a guard.

### STEP 4 Teach It Back

#### DEFINITION

Explain to a non-technical colleague: 'What is a recursive function and why is it useful for hierarchy data?' Use nesting dolls. Describe: (1) what recursion is without using the word; (2) what the smallest doll represents; (3) what happens without a smallest doll.

### Exercise 4 — Function Library — Design, Build, Deploy

■ 30 minutes

### STEP 1 Explain It First

### ■ Feynman Check

Before writing code:

- Name three concrete maintenance advantages of a library over copy-pasted functions.
- Can ETL[ValidateSchema] call ETL[SafeDate] inside its body? What constraint applies?
- 'Load to model' is enabled on ETL. What appears in the Power BI model? Is this correct?
- How do you add a Locale parameter to ETL[FormatCurrency] without breaking existing callers?

## STEP 2 Build It

### Task 4.1 — Build the Utils Library

Create blank query Utils (disabled from model load). Result: a record with Version='1.0' and these seven functions, all with full type annotations:

- **SafeDate(v)**: Returns nullable date. Never crashes.
- **SafeNumber(v)**: Returns nullable number. Never crashes.
- **NormaliseText(t)**: Trim + Text.Proper + collapse double spaces. Handles null.
- **FormatCurrency(n, opts)**: Record options: Decimals=2, Symbol='€'. Returns text. Handles null.
- **IsValidEmail(t)**: True if t has exactly one '@' and at least one '.' after it.
- **QualityGate(tbl,minRows,maxRows)**: Raises Error.Record if row count outside [minRows, maxRows]. maxRows optional nullable number.
- **ValidateSchema(tbl,required)**: Raises Error.Record listing missing columns if any absent.

■ **Hint** *IsValidEmail: let p = Text.Split(t,'@') in List.Count(p)=2 and Text.Contains(p{1},'.')*

■ **Hint** *QualityGate: N=Table.RowCount(tbl). Check N<minRows, then maxRows<>null and N>maxRows.*

■ **Hint** *Utils = [ Version='1.0', SafeDate=..., ... ] in Utils — the query result IS the record.*

■ **Hint** *Disable from model: right-click → Properties → uncheck 'Load enabled'.*

### ■ RESULT

Utils[Version] = '1.0'.

Utils[SafeDate]('2024-13-45') = null.

Utils[NormaliseText](' hello WORLD ') = 'Hello World'.

Utils[IsValidEmail]('user@domain.com') = true. Utils[IsValidEmail]('notanemail') = false.

Utils[QualityGate](#table({'A'},{{1},{2}}), 5, null) → Error.Record with N=2.

### Task 4.2 — Deploy the Library to a Sales ETL Pipeline

Create SalesETL. Source: inline #table with 10+ rows including 2 invalid dates, 1 null revenue, 1 negative revenue, 1 malformed email. Pipeline: ValidateSchema → SafeDate each date → SafeNumber each revenue → NormaliseText columns → FormatCurrency column → QualityGate → apply types.

■ **Hint** *Utils[ValidateSchema](RawData, {'CustomerID','Date','Revenue','Email'})*

■ **Hint** *Table.AddColumn(...,'ParsedDate', each Utils[SafeDate]([Date]), type nullable date)*

**Hint** `Utils[QualityGate](processed, 1, null)` — minimum 1 row, no maximum

### ■ RESULT

SalesETL uses only `Utils[...]` calls — no duplicated logic.  
 Invalid dates become null. Negative revenue handled. Email normalised.  
 Quality gate passes when row count  $\geq 1$ . All columns explicitly typed.

## Task 4.3 — Upgrade Utils to Version 1.1

Add `TruncateText(t, maxLen, optional suffix)` (suffix defaults to `'...'`). Update Version to `'1.1'`. Verify SalesETL works unchanged. Use `Utils[TruncateText]` in SalesETL for the email column (max 20 chars). Verify `Utils[Version] = '1.1'` from SalesETL without touching SalesETL.

**Hint** `suffix ?? '...'` — if caller omits suffix, use `'...'`.

### ■ RESULT

`Utils[Version] = '1.1'` readable from SalesETL without modifying SalesETL.  
`Utils[TruncateText]('hello@longdomain.example.com', 20) = 'hello@longdomain.ex...'`  
 Existing SalesETL steps unchanged and producing correct output.

## STEP 3 Break It

- Enable 'Load to model' on Utils. What appears in the Power BI Fields pane? What type does it show? Disable again.
- Add circular reference in Utils: `SafeDate` calls `ValidateSchema` and vice versa. What error does M raise?
- Rename Utils to `'ETL_Utils'`. What happens to SalesETL? What must you update?

## STEP 4 Teach It Back

### DEFINITION

Write a plain-English pitch to a team that copy-pastes transformation logic — three paragraphs: (1) the specific problem; (2) how the library pattern works; (3) the business cost of not fixing it, with a concrete threshold-change example.

## Exercise 5 — Advanced Patterns — Dispatch, Memoisation, Generic Transforms

■ 35 minutes

## STEP 1 Explain It First

### ■ Feynman Check

Before writing code:

- `Record.FieldOrDefault(Dispatchers, key, default)`: explain in one sentence without using `'dispatch'` or `'record'`.
- Naive recursive Fibonacci is  $O(2^n)$ . What does memoisation change about complexity?
- `SmartFormat` calls itself for list and record types. What operator enables this?
- `SafeRun(fn, arg)`: what does `'try fn(arg)'` return when `fn(arg)` succeeds? Fails?

## STEP 2 Build It

### Task 5.1 — Dynamic Dispatch Transform System

Build a configurable column transform system. Transformers record must have at least: Upper, Lower, Proper, Trim, TruncAt50, RemoveWhitespace. Config table: ColumnName, TransformerName. Write `ApplyConfig(src, config)` that reads config and applies the named transformer to each column. Adding a new transformer: one new Transformers field, nothing else.

**Hint** `ApplyConfig: List.Accumulate(Table.ToRecords(config), src, (acc,row) => Table.TransformColumns(acc, {{row[ColumnName], TransformerName}}))`

#### ■ RESULT

Config row {ColumnName='Name',TransformerName='Proper'}: Name column has Text.Proper applied.  
Unknown TransformerName falls back to identity — no crash.  
New transformer 'Reverse'=Text.Reverse in Transformers: works immediately in config.

### Task 5.2 — Memoised Lookup with Table.Buffer

`SlowLookup(n)` simulates an expensive computation (e.g. `List.Sum({1..n*1000})`). Pre-compute `n=1..100` into a buffered lookup table. Write `MemoLookup(n)` reading from the buffer. Prove the buffer is computed once by timing 100 `MemoLookup` calls vs 100 `SlowLookup` calls.

**Hint** `MemoTable = Table.Buffer(Table.FromRecords(List.Transform({1..100}, (n) => [Key=n, Value=SlowLookup(n)])))`

**Hint** `MemoLookup = (n) => Table.First(Table.SelectRows(MemoTable, each [Key]=n), null)?[Value]`

#### ■ RESULT

`MemoLookup(n) = SlowLookup(n)` for all `n` in `1..100`.  
100 `MemoLookup` calls: fast (reads from buffer). 100 `SlowLookup` calls: slow.  
`MemoTable`: 100 rows — one per pre-computed key.

### Task 5.3 — SmartFormat on a Complex Mixed-Type Table

Build `SmartFormat` handling: null, number, text, date, datetime, logical, list (recursively). Create an inline table with all these types plus a list column. Apply `SmartFormat` to every column using `Table.TransformColumns + List.Transform` on `ColumnNames`. Verify list column is converted to '[v1, v2, v3]' text.

**Hint** `Recursive list: ']' & Text.Combine(List.Transform(v, each @SmartFormat(_))& ', ' & '['`

#### ■ RESULT

`null`→'(null)'. `number`→'42'. `date`→'2024-01-15'. `datetime`→'2024-01-15 09:00'.  
`logical`→'Yes'/'No'. `list`→'[1, 2, 3]'. All columns type text after transformation.

### Task 5.4 — SafeRun Applied to a Bulk Enrichment Pipeline

Inline table of 6 IDs where 2 simulate 'not found' (lookup function throws on `ID > 1000`). Apply `SafeRun` to enrich each ID. Produce: `CleanTable` (`Success=true` rows) and `ErrorsTable` (`Success=false` rows with error messages). All 6 rows processed — no crash.

**Hint** `LookupID = (id) => if id > 1000 then error Error.Record('NotFound','ID not found',id) else [Name='User '&Text.From(id)]`

**■ Hint**

*CleanRows = Table.SelectRows(Expanded, each [OK]=true). ErrorRows = Table.SelectRows(Expanded, each [OK]=fal...*

**■ RESULT**

CleanTable: 4 rows with Name populated.

ErrorsTable: 2 rows with OK=false and Err messages.

Total = 6. No crash despite 2 failures.

**STEP 3**  
**Break It**

- Task 5.1: TransformerName not in Formatters. Is identity fallback the right design? When should you raise an error instead?
- Task 5.2: remove Table.Buffer from MemoTable. Add a second column also calling SlowLookup. How many times is SlowLookup called per key?
- Task 5.3: add a column whose values are lists of records. Does SmartFormat handle this? Trace the recursive calls.

**STEP 4**  
**Teach It Back — Module UDF Capstone****DEFINITION**

You are designing the UDF architecture for a new enterprise Power BI model with 15 queries and 4 developers. Write a plain-English architecture decision record — one paragraph per decision — covering: (1) how you organise shared functions and why; (2) type annotations policy and why; (3) how you handle versioning as the library evolves; (4) patterns for functions receiving null or bad input.

## Module Review and Takeaways

### Review Questions

**Q: What does first-class function status mean in M?**

**A:** Functions can be stored in variables, passed as arguments, returned as results, and collected in records and lists — identical to how numbers or text work.

**Q: What is the difference between optional and nullable in a parameter annotation?**

**A:** optional means the caller may omit the argument — M passes null if omitted. nullable number means the parameter accepts a number OR null explicitly. Use both together: optional param as nullable number. Inside the function both omitted and explicit null appear as null.

**Q: What is a closure?**

**A:** A function that captures variables from its enclosing lexical scope. In M, any function defined inside a let...in automatically captures all visible bindings. The captured value is fixed at creation time. Because M has no mutation, there are no shared-state closure bugs.

**Q: Why does M need the @ operator for recursive functions?**

**A:** Inside a let binding the name being defined is not in scope within its own body. @ is a special exception providing self-reference. Without it a function cannot call itself and M raises Expression.Error.

**Q: What is the record options pattern?**

**A:** A single optional opts as nullable record parameter merged with a Defaults record using Record.Combine({Defaults, opts ?? []}). Use it for 3+ optional parameters. Extensible (add one Defaults field), backwards-compatible (existing callers pass no opts and get all defaults).

**Q: What is dynamic dispatch and how is it implemented?**

**A:** Selecting and calling a function by name at runtime without if/else chains. Store functions in a record keyed by their names. Use Record.FieldOrDefault(Dispatchers, key, defaultFn) to select. O(1) lookup. Adding a new option requires only adding one record field.

**Q: When should you use recursion vs List.Generate?**

**A:** Recursion with @: tree traversal, hierarchy paths, naturally recursive math (GCD, factorial). Always include a depth limit. List.Generate: sequential iteration, counters, pagination, retry loops. No stack overflow risk. For depth > 100 always prefer List.Generate.

### Best Practices

- Always annotate parameter and return types on shared functions. Checked at call time; serves as machine-readable documentation.
- Use ?? for optional parameter defaults — shorter, clearer, correct for 0/false/". Not the same as SQL COALESCE.
- Record options pattern for 3+ optional parameters. Extensible without breaking callers.
- Function library in a single disabled-from-model query. All other queries call by name.
- Include Version and Help record in every library. Self-documenting libraries save debugging time.
- Always include a depth limit in recursive functions: if depth > N then error '...'. Test with circular data.

- Use @ explicitly for self-reference — never assume the name is in scope within its own body.
- Buffer tables before row-by-row reference inside Table.AddColumn or recursive functions.
- Dynamic dispatch: Record.FieldOrDefault + function records. Zero if/else in calling code.
- Memoisation: pre-compute into Table.Buffer or List.Buffer. Measure before/after to confirm benefit.
- SafeRun(fn, arg): try fn(arg) returning [OK, Value, Err]. Isolate failures per row.
- Type-safe generics: Value.Is for runtime type checks. Handle null explicitly before any type check.
- Factory functions are the M equivalent of partial application — define once, configure many times.
- Document before/after: every refactoring of copy-pasted code into a library must be verified to produce identical output.

## Common Mistakes

### ■ ■ Watch Out

#### Calling without parentheses

Cause: Double = (x)=>x\*2. Writing 'Double' references the function. Writing 'Double(5)' calls it.

Fix: Always verify: Result = Double(5) not Result = Double.

### ■ ■ Watch Out

#### Forgetting @ for self-reference

Cause: F = (n) => if n<=1 then 1 else n \* F(n-1) — F not in scope. Expression.Error at runtime.

Fix: Use @F: n \* @F(n-1). Always. No exceptions.

### ■ ■ Watch Out

#### No depth limit in recursive functions

Cause: Circular data causes infinite recursion → stack overflow → entire refresh crashes.

Fix: Always: if depth > N then error Error.Record('MaxDepth','...',depth). Test with circular data.

### ■ ■ Watch Out

#### ?? for type-unsafe defaults

Cause: 'false ?? 99' returns false. '0 ?? 99' returns 0. Correct but surprising if expecting SQL COALESCE behaviour.

Fix: ?? is null-coalescing only. For defaults on any falsy value use: if x = null then default else x.

### ■ ■ Watch Out

#### Buffering before fold-eligible steps

Cause: Table.Buffer before Table.SelectRows breaks query folding — SelectRows runs in M memory.

Fix: Buffer after the last fold-eligible step, not before. Module 9 rules apply.

 Watch Out**Circular references in library records**

Cause: ETL[A] calls ETL[B] and ETL[B] calls ETL[A]. Circular dependency error.

Fix: Extract shared logic to a private helper binding inside the library record, called by both.

## Quick Reference — User-Defined Functions

| Syntax / Pattern                | Category | Key Behaviour                                       |
|---------------------------------|----------|-----------------------------------------------------|
| (p) => expr                     | Define   | Anonymous single-parameter function                 |
| (p1, p2) => expr                | Define   | Anonymous multi-parameter function                  |
| (p as T) as R => expr           | Define   | Typed function; T=param type, R=return type         |
| optional p as nullable T        | Define   | Optional param; M passes null if caller omits it    |
| x ?? default                    | Param    | Null-coalescing: return default if x is null        |
| Record.Combine({Def,opts ??[]}) | Param    | Defaults-and-overrides merge pattern                |
| Value.Is(v, type T)             | Type     | Runtime type check; true if v conforms to T         |
| Value.Type(v)                   | Type     | Return the M type of value v                        |
| @FuncName(args)                 | Recurse  | Self-reference inside let. Required for recursion.  |
| if depth > N then error ...     | Recurse  | Depth limit — always include in recursive functions |
| Compose(f,g) = (x)=>g(f(x))     | HOF      | Function composition: f applied first, then g       |
| Pipeline(fns) via Accumulate    | HOF      | Compose a list of functions left-to-right           |
| Lib = [F1=..., F2=...] in Lib   | Library  | Function library record. Call: Lib[F1](args).       |
| Record.FieldOrDefault(r, k, d)  | Dispatch | Dynamic dispatch by name. O(1). No if/else.         |
| try fn(arg)                     | Error    | [HasError, Value/Error] record. Catch all errors.   |
| Table.Buffer(precomputed)       | Memo     | Memoisation: compute once, look up by key.          |
| (factory)(captured)(arg)        | Closure  | Factory → closure → call: MakeMult(3)(7) = 21       |
| each Record.Field(_, col)       | Dynamic  | Dynamic column access when name is a variable       |

### ■ PL-300 Exam

#### PL-300 UDF Exam Checklist:

- Syntax: (param as type) as returnType => body. Both as clauses are optional but recommended.
- optional + ??: optional param receives null if omitted; ?? supplies the default.
- Type annotations checked at call time — before the body executes.
- @ required for self-reference in recursive functions. Always include depth limit.
- Function library: blank query whose result is a record of functions. Call: LibName[FnName](args).
- Dynamic dispatch: Record.FieldOrDefault(Dispatchers, key, defaultFn). Zero if/else.
- Higher-order: Table.AddColumn, List.Transform, List.Select all take function arguments.
- Closures: inner functions capture outer let bindings. Factory functions return pre-configured closures.

#### ■ How M Does It

This module integrates directly into Module 8 (API connector function library), Module 9 (fold-safe parameterised filters using factory functions), and Module 10 (buffered memoisation for performance). The ETL library pattern, dynamic dispatch, and SafeRun wrapper are applied unchanged in production connector architecture.